

TalentGeenie — High-Level Design (HLD)

Document Version: 2.0
Date: March 5, 2026
Product: TalentGeenie — AI-Powered Interview & Learning Platform
Classification: Internal / Confidential

Table of Contents

- 1. [Introduction](#)
 - 2. [System Architecture Overview](#)
 - 3. [Technology Stack](#)
 - 4. [System Context Diagram](#)
 - 5. [Component Architecture](#)
 - 6. [Data Architecture](#)
 - 7. [Authentication & Authorization Architecture](#)
 - 8. [AI Architecture](#)
 - 9. [Infrastructure & Deployment Architecture](#)
 - 10. [Integration Architecture](#)
 - 11. [Key Design Decisions](#)
 - 12. [Scalability Strategy](#)
 - 13. [Security Architecture](#)
-

1. Introduction

1.1 Purpose

This document presents the High-Level Design (HLD) for the TalentGeenie platform. It describes the system architecture, major components, their interactions, and the key design decisions that shape the platform.

1.2 Scope

TalentGeenie is a multi-tenant SaaS platform comprising:

- **Frontend:** React SPA with hybrid mobile app support (Capacitor)
- **Backend:** Self-hosted Supabase (10 Docker services)
- **AI Engine:** Google Gemini integration for question generation and evaluation
- **Proctoring Engine:** Browser-based video/audio monitoring with AI analysis

1.3 References

Document	Location
Requirements (SRS)	docs/REQUIREMENTS_DOCUMENT.md
Low-Level Design (LLD)	docs/LOW_LEVEL_DESIGN.md
Security Audit	SECURITY_AUDIT_COMPLETE.md

2. System Architecture Overview

2.1 Architecture Style

TalentGeenie follows a **modular monolith frontend + serverless backend** architecture:

- **Frontend:** Single React SPA handling all UI concerns
- **Backend:** Supabase platform providing authentication, database, storage, and edge functions
- **API Gateway:** Kong proxies all API requests
- **Database:** PostgreSQL with RLS for multi-tenant data isolation

2.2 High-Level Architecture Diagram

```
flowchart TD
    subgraph Client["CLIENT TIER"]
        Web["Web Browser<br/>(React SPA)"]
        iOS["iOS App<br/>(Capacitor)"]
        Android["Android App<br/>(Capacitor)"]
    end

    Web & iOS & Android -->| "HTTPS / WSS" | Kong

    subgraph Gateway["API GATEWAY TIER"]
        Kong["Kong Gateway<br/>Port 8000<br/>Rate Limiting, CORS, Auth Header Forwarding"]
        Kong --> Auth["/auth - GoTrue"]
        Kong --> REST["/rest - PostgREST"]
        Kong --> Func["/functions - Deno"]
    end

    Auth & REST & Func --> PgBouncer

    subgraph Backend["BACKEND TIER"]
        PgBouncer["PgBouncer :6432<br/>Connection Pooling (1000 max)"]
        PgBouncer --> PG["PostgreSQL 15.8 :54322<br/>60+ Tables · 406 RLS · 123 Functions"]
        Realtime["Realtime<br/>(WebSocket)"]
        Storage["Storage<br/>(S3-compatible)"]
        Imgproxy["Imgproxy<br/>(Image Trans.)"]
        Studio["Studio<br/>(Admin UI)"]
    end

    Func -->|HTTPS| Gemini["Google Gemini API<br/>AI Generation & Evaluation"]
    Func -->|HTTPS| Stripe["Stripe API<br/>Payments"]
    Func -->|SMTP| SMTP["SMTP Server<br/>Email Delivery"]

    style Client fill:#fff6ff,stroke:#3b82f6,stroke-width:2px
    style Gateway fill:#fef3c7,stroke:#f59e0b,stroke-width:2px
    style Backend fill:#dcfce7,stroke:#22c55e,stroke-width:2px
    style Gemini fill:#f3e8ff,stroke:#a855f7
    style Stripe fill:#fce7f3,stroke:#ec4899
    style SMTP fill:#e0f2fe,stroke:#0ea5e9
```

3. Technology Stack

3.1 Frontend Stack

```
block-beta
  columns 3
  block:FE["FRONTEND STACK"]:3
    columns 3
    block:React["React 18.3 (SPA)":3
      columns 3
      TS["TypeScript 5.8"]
      RR["React Router DOM 6.30"]
      RQ["React Query (TanStack) 5"]
      RHF["React Hook Form 7.61"]
      Zod["Zod 4.1"]
      SUI["shadcn/ui (50+ comp.)"]
    end
    TW["Tailwind CSS 3.4"]
    RC["Recharts 2.15"]
    ME["Monaco Editor 4.7"]
    Cap["Capacitor 7.4 (Mobile)"]
    Sen["Sentry 10.39"]
    MP["MediaPipe + HuggingFace"]
  end

style FE fill:#eff6ff,stroke:#3b82f6,stroke-width:2px
style React fill:#dbeafe,stroke:#3b82f6
```

Build: Vite 5.4 + SWC

3.2 Backend Stack

```
flowchart LR
  subgraph Supabase["SELF-HOSTED SUPABASE – 10 Docker Services"]
    direction TB
    subgraph Row1[" "]
      direction LR
      PG["PostgreSQL 15.8"]
      PGB["PgBouncer<br/>(Pooling)"]
      K["Kong 2.8<br/>(Gateway)"]
    end
    end
    subgraph Row2[" "]
      direction LR
      GT["GoTrue<br/>(Auth)"]
      PR["PostgREST<br/>(REST API)"]
      RT["Realtime<br/>(WebSocket)"]
    end
    end
    subgraph Row3[" "]
      direction LR
      ST["Storage<br/>(Files)"]
    end
  end
```

```

        FN["Functions<br/>(107 Deno)"]
        IP["Imgproxy<br/>(Images)"]
        SU["Studio<br/>(Admin)"]
    end
end

style Supabase fill:#dcfce7,stroke:#22c55e,stroke-width:2px

```

4. System Context Diagram

```

flowchart LR
    PA["Platform Admin<br/>(Full Access)"] --> TG
    PtA["Partner Admin<br/>(Org Admin)"] --> TG
    HR["HR Recruiter<br/>(Interviews)"] --> TG
    TS["Tech SPOC<br/>(Questions)"] --> TG
    BG["Billing &<br/>Guest Users"] --> TG

    subgraph TG ["TalentGeenie Platform"]
        direction TB
        Apps["Web SPA · Mobile Apps"]
        SB["Self-Hosted Supabase Backend"]
        Apps --> SB
    end

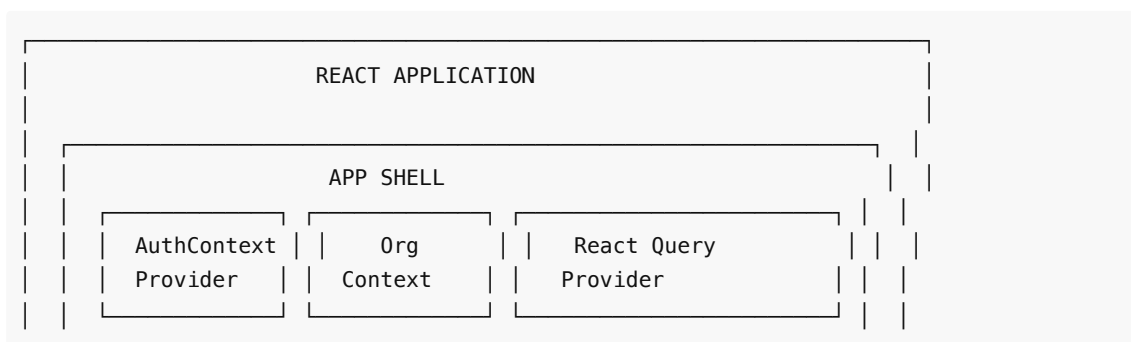
    TG --> Cand["Candidate<br/>(Interview Taker)"]
    TG -->|HTTPS| Gemini["Google Gemini<br/>(AI)"]
    TG -->|HTTPS| Stripe["Stripe API<br/>(Payments)"]
    TG -->|SMTP| SMTP["SMTP Server<br/>(Email)"]

    style TG fill:#eff6ff,stroke:#3b82f6,stroke-width:2px
    style Gemini fill:#f3e8ff,stroke:#a855f7
    style Stripe fill:#fce7f3,stroke:#ec4899
    style SMTP fill:#e0f2fe,stroke:#0ea5e9
    style Cand fill:#d1fae5,stroke:#10b981

```

5. Component Architecture

5.1 Frontend Module Decomposition



ErrorBoundary
(Sentry)

Sonner
Toaster

React Router
BrowserRouter

LAYOUTS

AppLayout
(Base)

AdminLayout
(/admin/*)

PartnerLayout
(/partner/*)

Shared: AppNavbar | RoleBreadcrumbs | ImpersonationBanner

FEATURE MODULES

INTERVIEW MODULE

- CreateInterview
- InterviewDetail
- Dashboard
- TakeInterview
- QuickCreate
- JDBuilder
- Templates
- QuestionRepo

PROCTORING MODULE

- PreInterviewCheck
- ProctoringMonitor
- ProctoringReport
- LiveStreamViewer
- VideoPlayer
- Integrity Checks
- 13 components

LEARNING MODULE

- LearningDash
- MyLearningPlan
- Certifications
- TakeCert
- MyCertificates
- LearningHistory
- CertVerify

ADMIN MODULE

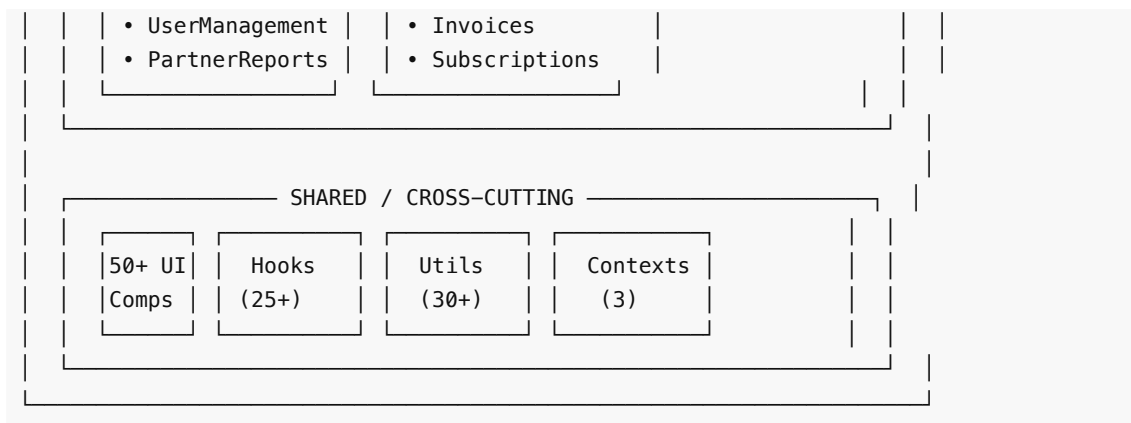
- AdminHub
- UserManagement
- OrgManagement
- AIConfiguration
- BillingMgmt
- SystemMonitoring
- 30+ pages

PARTNER MODULE

- PartnerPortal
- OrgSettings
- OrgAnalytics

BILLING MODULE

- PartnerBilling
- PlanManagement
- PaymentGateway



5.2 Backend Service Architecture

```

flowchart TD
    subgraph SB["SUPABASE BACKEND"]
        direction TB
        subgraph EF["EDGE FUNCTIONS – 107 Deno Functions"]
            direction TB
            subgraph EFRow1[" "]
                direction LR
                AI["AI Functions (11)<br/>• generate-qs<br/>• evaluate<br/>• generate-jd<br/>• extract-skills"]
                UF["User Functions (4)<br/>• user-signup<br/>• admin-mgmt<br/>• org-user<br/>• password"]
            end
            subgraph EFRow2[" "]
                direction LR
                PF["Proctoring (15)<br/>• init-session<br/>• log-violation<br/>• upload-video<br/>• merge-chunks"]
                EN["Email/Notif (10)<br/>• send-email<br/>• send-invite<br/>• send-notif<br/>• email-hook"]
            end
            subgraph EFRow3[" "]
                direction LR
                LF["Learning (6)<br/>• eval-learning<br/>• eval-cert<br/>• gen-questions"]
                BF["Billing (4)<br/>• gen-invoice<br/>• approve-app<br/>• delete-org"]
            end
            subgraph EFRow4[" "]
                direction LR
                TF["Testing (14)<br/>• run-tests<br/>• seed-data<br/>• cleanup"]
                PLF["Platform (12)<br/>• chatbot<br/>• scheduled-jobs<br/>• ats-webhook"]
            end
        end
        subgraph DB["DATABASE LAYER"]
            PG["PostgreSQL 15.8<br/>60+ Tables · 406 RLS Policies · 123 DB Functions<br/>Enums: app_role (12 values)<br/>Triggers: auto-update timestamps, cascading deletes<br/>Indexes: Performance-optimized queries"]
        end
    end
  
```

```

        end
    end

    style SB fill:#dcfce7,stroke:#22c55e,stroke-width:2px
    style EF fill:#eff6ff,stroke:#3b82f6,stroke-width:2px
    style DB fill:#fef3c7,stroke:#f59e0b,stroke-width:2px

```

6. Data Architecture

6.1 Entity-Relationship Overview

```

erDiagram
    AUTH_USERS ||--|| PROFILES : "1:1"
    PROFILES ||--o{ USER_ROLES : "1:N"
    PROFILES }o--o{ ORGANIZATION_MEMBERS : "N:N via"
    ORGANIZATION_MEMBERS }o--|| ORGANIZATIONS : "N:1"
    ORGANIZATIONS ||--o{ INTERVIEWS : "1:N"
    ORGANIZATIONS ||--o{ ORG_SUBSCRIPTIONS : "1:N"
    ORGANIZATIONS ||--o{ ACTIVITY_FEED : "1:N"
    ORG_SUBSCRIPTIONS }o--|| SUBSCRIPTION_PLANS : "N:1"
    INTERVIEWS ||--o{ QUESTIONS : "1:N"
    INTERVIEWS ||--o{ INVITATIONS : "1:N"
    INTERVIEWS ||--o{ ATTEMPTS : "1:N"
    INTERVIEWS ||--o{ PROCTORING_SESSIONS : "1:N"
    ATTEMPTS ||--|| ASSESSMENTS : "1:1 AI eval"

    AUTH_USERS {
        uuid id PK
        text email
    }
    PROFILES {
        uuid id PK
        text email
        text full_name
    }
    ORGANIZATIONS {
        uuid id PK
        text name
        text slug
    }
    INTERVIEWS {
        uuid id PK
        text title
        text status
    }
    ATTEMPTS {
        uuid id PK
        text candidate_email
        text status
    }

```

```

ASSESSMENTS {
    uuid id PK
    numeric overall_score
    text hiring_decision
}

```

6.2 Data Domain Groups

```

flowchart TD
    subgraph DD["DATA DOMAINS"]
        direction TB
        subgraph DRow1[" "]
            direction LR
            ID["IDENTITY<br/>• profiles<br/>• user_roles<br/>• custom_roles<br/>• user_custom_roles<br/>• role_permissions"]
            ORG["ORGANIZATION<br/>• organizations<br/>• org_members<br/>• org_subscriptions<br/>• activity_feed<br/>• onboarding"]
        end
        subgraph DRow2[" "]
            direction LR
            IV["INTERVIEW<br/>• interviews<br/>• questions<br/>• invitations<br/>• attempts<br/>• attempt_questions<br/>• assessments"]
            PR["PROCTORING<br/>• proctoring_sessions<br/>• preinterview_check_logs<br/>• candidate_performance_idx"]
        end
        subgraph DRow3[" "]
            direction LR
            LN["LEARNING<br/>• learning_assessments<br/>• learning_qs / learning_attempts<br/>• training_plans / training_topics<br/>• learning_mats<br/>• cert_topics / cert_questions<br/>• cert_attempts / user_certs"]
            BL["BILLING<br/>• subscription_plans<br/>• invoices<br/>• usage_tracking<br/>• payment_gw<br/>• payment_methods<br/>• payment_txns"]
        end
        subgraph DRow4[" "]
            direction LR
            AIC["AI CONFIG<br/>• ai_providers<br/>• ai_credentials<br/>• ai_model_config<br/>• ai_feature_conf<br/>• ai_health<br/>• ai_usage_logs"]
            PL["PLATFORM<br/>• platform_config<br/>• system_config<br/>• audit_logs<br/>• security_events<br/>• notifications<br/>• chatbot_knowledge"]
        end
    end

    style DD fill:#eff6ff,stroke:#3b82f6,stroke-width:2px
    style ID fill:#dcfce7,stroke:#22c55e
    style ORG fill:#fef3c7,stroke:#f59e0b
    style IV fill:#f3e8ff,stroke:#a855f7
    style PR fill:#fce7f3,stroke:#ec4899
    style LN fill:#e0f2fe,stroke:#0ea5e9
    style BL fill:#fef9c3,stroke:#eab308

```

```

style AIC fill:#f3e8ff,stroke:#a855f7
style PL fill:#dcfce7,stroke:#22c55e

```

6.3 Multi-Tenant Data Isolation Model

```

flowchart TD
    subgraph MT["MULTI-TENANT ISOLATION"]
        direction TB
        subgraph Orgs[" "]
            direction LR
            subgraph OrgA["Organization A"]
                A1["interviews ← RLS Filter"]
                A2["questions ← RLS Filter"]
                A3["attempts ← RLS Filter"]
                A4["members ← RLS Filter"]
            end
            subgraph OrgB["Organization B"]
                B1["interviews ← RLS Filter"]
                B2["questions ← RLS Filter"]
                B3["attempts ← RLS Filter"]
                B4["members ← RLS Filter"]
            end
        end
        RLS["RLS Policy Pattern:<br/><br/>CREATE POLICY org_isolation ON<br/>interviews<br/>USING (<br/> organization_id IN (<br/> SELECT organization_id<br/>FROM organization_members<br/> WHERE user_id = auth.uid AND is_active = true<br/>)<br/> OR has_role auth.uid, platform_admin<br/>")
        PA["Platform Admin: Bypasses all org isolation (god mode)"]
    end

    style MT fill:#fef3c7,stroke:#f59e0b,stroke-width:2px
    style OrgA fill:#dcfce7,stroke:#22c55e
    style OrgB fill:#eff6ff,stroke:#3b82f6
    style RLS fill:#f3e8ff,stroke:#a855f7
    style PA fill:#fce7f3,stroke:#ec4899

```

7. Authentication & Authorization Architecture

7.1 Authentication Flow

```

sequenceDiagram
    participant U as User
    participant SPA as SPA
    participant GT as GoTrue
    participant EF as Edge Function
    participant DB as Database

    U->>SPA: 1. Enter Email
    SPA->>DB: 2. check_user_exists

```

```

DB-->>SPA: User status

U->>SPA: 3. Enter Password
SPA-->>GT: 4. signInWithPassword
GT-->>SPA: 5. JWT + Session

SPA-->>EF: 6. complete-user-signup (if new)
EF-->>DB: 7. Create Profile
EF-->>DB: 8. Assign Guest Role
DB-->>EF: OK
EF-->>SPA: Profile created

SPA-->>DB: 9. Fetch user_roles
DB-->>SPA: Roles array

SPA-->>U: 10. Redirect to Role Dashboard

```

7.2 Authorization Layer Architecture

```

flowchart TD
    subgraph AUTH["4-LAYER AUTHORIZATION"]
        direction TB
        subgraph L1["Layer 1: ROUTE LEVEL"]
            R1["ProtectedRoute requiredRoles=platform_admin<br/>→ AdminLayout → Route"]
        end
        subgraph L2["Layer 2: LAYOUT LEVEL"]
            R2["AdminLayout → platform_admin only<br/>PartnerLayout → partner_admin, hr_recruiter,<br/>tech_spoc, billing_contact, platform_admin"]
        end
        subgraph L3["Layer 3: COMPONENT LEVEL"]
            R3["PermissionGate permission=manage_billing<br/>→ BillingSection"]
        end
        subgraph L4["Layer 4: DATABASE LEVEL – RLS"]
            R4["406 Row-Level Security Policies<br/>• Table-level read/write restrictions<br/>• Organization-scoped data access<br/>• Role-based query filtering<br/>• SECURITY DEFINER functions for sensitive ops"]
        end
    end

    L1 --> L2 --> L3 --> L4

    style AUTH fill:#eff6ff,stroke:#3b82f6,stroke-width:2px
    style L1 fill:#dcfce7,stroke:#22c55e
    style L2 fill:#fef3c7,stroke:#f59e0b
    style L3 fill:#fce7f3,stroke:#ec4899
    style L4 fill:#f3e8ff,stroke:#a855f7

```

7.3 Role Hierarchy

```

flowchart TD
    PA["PLATFORM ADMIN<br/>(God Mode)<br/><br/>All permissions<br/>All organizations<br/>All data access"]
    PA --> PtA["PARTNER ADMIN<br/><br/>Own org management<br/>Users, billing,<br/>recruiting, settings"]
    PtA --> HR["HR RECRUITER<br/><br/>Create/share interviews,<br/>evaluate, proctoring"]
    PtA --> TS["TECH SPOC<br/><br/>Questions, templates,<br/>read-only interviews"]
    PtA --> BC["BILLING CONTACT<br/><br/>Billing, invoices,<br/>subscriptions"]
    HR --> G["GUEST<br/><br/>Learning dashboard,<br/>certifications only<br/>Awaiting role assignment"]

    style PA fill:#ef4444,color:#fff,stroke:#b91c1c,stroke-width:2px
    style PtA fill:#f59e0b,color:#fff,stroke:#d97706,stroke-width:2px
    style HR fill:#3b82f6,color:#fff,stroke:#2563eb
    style TS fill:#22c55e,color:#fff,stroke:#16a34a
    style BC fill:#a855f7,color:#fff,stroke:#9333ea
    style G fill:#6b7280,color:#fff,stroke:#4b5563

```

8. AI Architecture

8.1 AI Integration Architecture

```

flowchart TD
    subgraph AIArch ["AI ARCHITECTURE"]
        direction TB
        subgraph FE ["Frontend"]
            CI["CreateInterview"]
            TI["TakeInterview<br/>(post-submit)"]
        end
        end
        subgraph EFunc ["Edge Functions"]
            GQ["generate-questions<br/>1. Parse JD & Config<br/>2. Build AI Prompt<br/>3. Call Gemini 2.5 Flash<br/>4. Parse Response<br/>5. Validate Questions<br/>6. Store in DB"]
            EI["evaluate-interview<br/>1. Fetch Answers<br/>2. Build Eval Prompt<br/>3. Call Gemini 2.5 Flash<br/>4. Score per Question<br/>5. Calculate CPI<br/>6. Hiring Decision<br/>7. Store Assessment"]
        end
        end
        CI -->|HTTP| GQ
        TI -->|HTTP| EI
        GQ & EI -->|HTTPS| GEM
        subgraph GEM ["Google Gemini API"]
            GP["Primary: Gemini 2.5 Flash"]
            GF["Fallback: Gemini 1.5 Flash"]
        end
        end
        subgraph CSA ["Client-Side AI"]
            MP["MediaPipe Tasks Vision – Face Detection, Gaze Tracking"]
            HF["HuggingFace Transformers – Object Detection"]
            CL["Custom Libraries – Voice Analysis, Liveness Detection"]
        end

```

```

        Note1["Usage: Proctoring – lightweight during interview,<br/>heavy
analysis deferred to post-interview"]
    end
end

```

```

style AIArch fill:#f3e8ff,stroke:#a855f7,stroke-width:2px
style FE fill:#eff6ff,stroke:#3b82f6
style EFunc fill:#dcfce7,stroke:#22c55e
style GEM fill:#fef3c7,stroke:#f59e0b
style CSA fill:#fce7f3,stroke:#ec4899

```

8.2 AI Feature Mapping

```

flowchart TD
    subgraph AIF["AI FEATURES & MODELS"]
        direction TB
        subgraph AIRow1[" "]
            direction LR
            QG["QUESTION GENERATION<br/>Model: Gemini 2.5 Flash<br/><br/>• generate-
questions<br/>• generate-learning-qs<br/>• generate-cert-qs<br/>• regenerate-
questions<br/>• batch-regenerate"]
            EV["EVALUATION<br/>Model: Gemini 2.5 Flash<br/><br/>• evaluate-
interview<br/>• evaluate-learning<br/>• evaluate-cert<br/>• detect-bias<br/>•
calculate-cpi"]
        end
        subgraph AIRow2[" "]
            direction LR
            JD["JD GENERATION<br/>Model: Gemini 2.5 Flash<br/><br/>• generate-job-
desc<br/>• enhance-job-desc<br/>• suggest-jd-content<br/>• extract-skills<br/>•
parse-resume"]
            PRC["PROCTORING AI<br/>(Client-Side)<br/><br/>• Face Detection<br/>•
Gaze Tracking<br/>• Object Detection<br/>• Voice Analysis<br/>• Liveness Check"]
        end
        subgraph AIRow3[" "]
            direction LR
            AS["ASSISTANCE<br/>Model: Configurable<br/><br/>• chatbot-assist<br/>•
enhance-email-content<br/>• log-analysis-chat"]
            MO["MONITORING<br/><br/>• ai_health_monitor<br/>• ai_usage_logs<br/>•
ai_feature_health<br/>• A/B testing"]
        end
    end
end

```

```

style AIF fill:#f3e8ff,stroke:#a855f7,stroke-width:2px
style QG fill:#dcfce7,stroke:#22c55e
style EV fill:#fef3c7,stroke:#f59e0b
style JD fill:#eff6ff,stroke:#3b82f6
style PRC fill:#fce7f3,stroke:#ec4899
style AS fill:#e0f2fe,stroke:#0ea5e9
style MO fill:#fef9c3,stroke:#eab308

```

9. Infrastructure & Deployment Architecture

9.1 Docker Compose Architecture (Self-Hosted)

```
flowchart TD
    subgraph DCS["DOCKER COMPOSE STACK"]
        direction TB
        subgraph SB["docker-compose.supabase.yml - 10 services"]
            Kong["Kong - API Gateway<br/>Port: 8000"]
            Kong --> GoTrue["GoTrue - Auth<br/>:9999"]
            Kong --> PostgREST["PostgREST - REST"]
            Kong --> Functions["Functions - 107 Deno"]
            GoTrue & PostgREST & Functions --> PgBouncer["PgBouncer - Pooling<br/>Port: 6432<br/>Max Clients: 1000"]
            PgBouncer --> PG["PostgreSQL 15.8<br/>Port: 54322<br/>Volume: pgdata - persistent"]
            Realtime["Realtime - WS"]
            Storage["Storage - Files"]
            Imgproxy["Imgproxy - Images"]
            Studio["Studio - Admin<br/>:3000"]
        end
        end
        subgraph FEStack["docker-compose.yml - Frontend"]
            Nginx["Nginx - Static SPA Serving<br/>Port: 8080<br/>Non-root user<br/>Gzip, Security Headers, SPA Routing"]
        end
    end

    style DCS fill:#eff6ff,stroke:#3b82f6,stroke-width:2px
    style SB fill:#dcfce7,stroke:#22c55e
    style FEStack fill:#fef3c7,stroke:#f59e0b
```

9.2 Production Deployment Architecture

```
flowchart TD
    subgraph PD["PRODUCTION DEPLOYMENT"]
        direction TB
        subgraph K8S["Kubernetes Cluster"]
            FEDep["Frontend Deployment<br/>(Nginx)"]
            SBSvc["Supabase Services<br/>(10 pods)"]
            EFDep["Edge Functions<br/>(Deno Workers)"]
            SEC["Secrets (K8s)"]
            CM["ConfigMaps (Settings)"]
            HELM["Helm Charts for<br/>templated deployment"]
        end
        end
        subgraph PS["Persistent Storage"]
            PGV["PostgreSQL Volume"]
            STB["Storage Buckets"]
            REC["Recording Files"]
        end
    end
```

```

    subgraph EXT["External"]
        GEM["Google Gemini"]
        STR["Stripe"]
        SMTP["SMTP"]
        SEN["Sentry"]
    end
end

K8S --> PS
K8S --> EXT

style PD fill:#eff6ff,stroke:#3b82f6,stroke-width:2px
style K8S fill:#dcfce7,stroke:#22c55e
style PS fill:#fef3c7,stroke:#f59e0b
style EXT fill:#f3e8ff,stroke:#a855f7

```

9.3 Build Pipeline

```

flowchart LR
    SC["Source Code<br/><br/>TypeScript<br/>React<br/>Tailwind"] --> BUILD["Build - Vite<br/><br/>• tsc<br/>• Vite<br/>• SWC<br/>• dist/"]
    BUILD --> CONT["Container - Docker<br/><br/>• Multi-stage<br/>• Nginx<br/>• :8080"]
    CONT --> DEP["Deploy - K8s<br/><br/>• Helm<br/>• kubectl<br/>• Secrets"]

    style SC fill:#eff6ff,stroke:#3b82f6
    style BUILD fill:#fef3c7,stroke:#f59e0b
    style CONT fill:#dcfce7,stroke:#22c55e
    style DEP fill:#f3e8ff,stroke:#a855f7

```

10. Integration Architecture

10.1 External System Integrations

```

flowchart TD
    subgraph INT["INTEGRATION POINTS"]
        direction TB
        subgraph AIP["AI Provider"]
            AI1["Edge Function --HTTPS--> Google Gemini API<br/><br/>• API Key: Encrypted via AES in ai_provider_credentials<br/>• Retry: 3 attempts, exponential backoff<br/>• Fallback: Gemini 1.5 Flash if 2.5 Flash fails<br/>• Monitoring: ai_health_monitoring, ai_usage_logs"]
        end
        subgraph PAY["Payment Provider"]
            P1["Edge Function --HTTPS--> Stripe API<br/><br/>• Config: payment_gateways table - test/live mode<br/>• Methods: payment_methods - card brand, last4<br/>• Webhooks: Stripe -> edge function -> DB update<br/>• Keys: Encrypted, never client-exposed"]
        end
    end

```

```

    subgraph EM["Email Provider"]
      E1["Edge Function --SMTP--> Email Server<br/><br/>• Templates:
email_templates table<br/>• Types: Welcome, Verification, Invitation,<br/>
Reminder, Password Reset, Review Request<br/>• AI Enhancement: Optional AI-improved
content"]
    end
    subgraph ATS["ATS Integration"]
      AT1["Webhook: ATS Provider --HTTPS--> ats-webhook function<br/>Sync:
sync-ats-candidates function → ats_candidates<br/>• Config: ats_integrations table"]
    end
    subgraph ERR["Error Monitoring"]
      ER1["React ErrorBoundary --HTTPS--> Sentry<br/><br/>• Section tags:
admin, partner, recruiting, assessment<br/>• Breadcrumbs: navigation events"]
    end
  end

  style INT fill:#eff6ff,stroke:#3b82f6,stroke-width:2px
  style AIP fill:#f3e8ff,stroke:#a855f7
  style PAY fill:#fce7f3,stroke:#ec4899
  style EM fill:#e0f2fe,stroke:#0ea5e9
  style ATS fill:#dcfcae,stroke:#22c55e
  style ERR fill:#fef3c7,stroke:#f59e0b

```

11. Key Design Decisions

11.1 Architecture Decisions

#	Decision	Rationale	Trade-offs
AD-01	Self-hosted Supabase instead of cloud	Full data sovereignty, no vendor lock-in, customizable	Higher operational overhead
AD-02	React SPA instead of SSR (Next.js)	Simpler deployment (static via Nginx), Capacitor mobile support	No server-side rendering (SEO via meta tags)
AD-03	Edge Functions (Deno) for business logic	Serverless scaling, language alignment (TypeScript), Supabase-native	Cold start latency for first calls
AD-04	PostgreSQL RLS for multi-tenant isolation	Database-level security, no application-level leaks	Query performance overhead per policy
AD-05	Client-side AI for proctoring	Reduced server load, real-time detection, no video streaming to server	Higher client CPU usage
AD-06	Deferred heavy AI analysis to post-interview	Better interview experience (no lag during exam)	Delayed proctoring reports
AD-07	Kong API Gateway	Centralized auth header forwarding, rate limiting, CORS	Configuration complexity

AD-08	PgBouncer for connection pooling	Handle 1000+ concurrent connections	Prepared statement limitations
AD-09	React Query for server state	Automatic caching, deduplication, optimistic updates	Learning curve, cache invalidation
AD-10	shadcn/ui component library	Customizable, accessible, no runtime dependency	Manual component management

11.2 Data Design Decisions

#	Decision	Rationale
DD-01	Session tokens (64-char) instead of incrementing IDs	Prevents attempt enumeration attacks
DD-02	SECURITY DEFINER functions for sensitive queries	Bypass RLS safely for candidate-facing queries
DD-03	AES encryption for API keys	Protect credentials at rest
DD-04	Immutable PII after attempt creation	Prevent post-submission tampering
DD-05	Separate correct_answer column with RLS	Candidates never see answers, even in API responses

12. Scalability Strategy

12.1 Horizontal Scaling Points

```
flowchart TD
    subgraph SA["SCALABILITY ARCHITECTURE"]
        direction TB
        subgraph S0["Stateless – Scale Out"]
            S1["Frontend – Nginx → Add replicas behind LB"]
            S2["Edge Functions – Deno → Horizontal auto-scale"]
            S3["PostgREST → Multiple instances"]
            S4["GoTrue – Auth → Multiple instances"]
            S5["Kong – Gateway → Multiple instances"]
        end
        subgraph SU["Stateful – Scale Up / Optimize"]
            U1["PostgreSQL → Read replicas, partitioning"]
            U2["PgBouncer → Increase pool size"]
            U3["Storage → S3-compatible backend"]
        end
        subgraph CL["Caching Layers"]
            C1["Client: React Query – in-memory, stale-while-revalidate"]
            C2["CDN: Nginx static asset cache – 1-year"]
            C3["API: Kong response caching – configurable"]
        end
    end
```

```

end
subgraph P0["Performance Optimizations"]
  P1["• Lazy loading for heavy pages – proctoring, editors"]
  P2["• Code splitting via React.lazy + Suspense"]
  P3["• Gzip compression on all text responses"]
  P4["• Chunked uploads for large recordings"]
  P5["• Background upload handler for reliability"]
end
end

style SA fill:#eff6ff,stroke:#3b82f6,stroke-width:2px
style S0 fill:#dcfce7,stroke:#22c55e
style SU fill:#fef3c7,stroke:#f59e0b
style CL fill:#f3e8ff,stroke:#a855f7
style P0 fill:#fce7f3,stroke:#ec4899

```

13. Security Architecture

13.1 Security Layers

```

flowchart TD
  subgraph DD["DEFENSE IN DEPTH"]
    direction TB
    subgraph SL1["Layer 1: NETWORK"]
      N1["• HTTPS/TLS 1.2+ – all traffic"]
      N2["• Kong API Gateway – rate limiting, CORS"]
      N3["• Nginx security headers – CSP, HSTS, X-Frame-Options"]
      N4["• Non-root container execution"]
    end
    subgraph SL2["Layer 2: AUTHENTICATION"]
      A1["• GoTrue – bcrypt passwords, JWT sessions"]
      A2["• Email verification required"]
      A3["• Automatic token refresh – 60s buffer"]
      A4["• Session token validation – 64-char per attempt"]
    end
    subgraph SL3["Layer 3: AUTHORIZATION"]
      Z1["• 6 hierarchical roles, 38 granular permissions"]
      Z2["• Route-level ProtectedRoute component"]
      Z3["• Component-level PermissionGate"]
      Z4["• 406 RLS policies on all database tables"]
      Z5["• SECURITY DEFINER functions for sensitive ops"]
    end
    subgraph SL4["Layer 4: DATA PROTECTION"]
      D1["• AES encryption for API keys at rest"]
      D2["• Correct answers hidden from candidates via RLS"]
      D3["• PII immutability after attempt creation"]
      D4["• Service role key restricted to edge functions"]
      D5["• Error message sanitization – no PG errors to client"]
    end
    subgraph SL5["Layer 5: MONITORING & AUDIT"]

```

```
    M1["• Sentry error monitoring with section tags"]
    M2["• Audit logs for admin actions"]
    M3["• Security event logs"]
    M4["• AI health and usage monitoring"]
    M5["• Pre-interview check logging"]
  end
end
```

```
SL1 --> SL2 --> SL3 --> SL4 --> SL5
```

```
style DD fill:#eff6ff,stroke:#3b82f6,stroke-width:2px
style SL1 fill:#dcfce7,stroke:#22c55e
style SL2 fill:#fef3c7,stroke:#f59e0b
style SL3 fill:#f3e8ff,stroke:#a855f7
style SL4 fill:#fce7f3,stroke:#ec4899
style SL5 fill:#e0f2fe,stroke:#0ea5e9
```